
IFM 3.1: Compilerbau (Winter 2026/27)

BC George & Carsten Gips (HSBI)

4. September 2026

Inhaltsverzeichnis

I. Syllabus	1
1. Syllabus	2
1.1. Kursbeschreibung	2
1.2. Überblick Modulinhalte	2
1.3. Team	3
1.4. Kursformat	3
1.5. Fahrplan	3
1.6. Prüfungsform, Note und Credits	4
1.7. Materialien	5
1.8. Förderungen und Kooperationen	5
1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada)	5
1.9. LICENSE	6
II. Vorlesungsunterlagen	7
2. Überblick	8
3. Lexikalische Analyse	9
3.1. Reguläre Sprachen, Ausdrucksstärke (Teil 1)	9
3.1.1. Motivation	9
3.1.2. Endliche Automaten	9
3.1.3. Reguläre Sprachen	11
3.1.4. Wrap-Up	14
3.2. Reguläre Sprachen, Ausdrucksstärke (Teil 2)	14
3.2.1. Wiederholung	14
3.2.2. Motivation	15
3.2.3. Lexer	15
3.2.4. Wrap-Up	16
4. Syntaktische Analyse	17
4.1. CFG	17
4.1.1. Wiederholung	17
4.1.2. Motivation	17
4.1.3. Kontextfreie Grammatiken und Sprachen	19
4.1.4. Syntaxanalyse	20

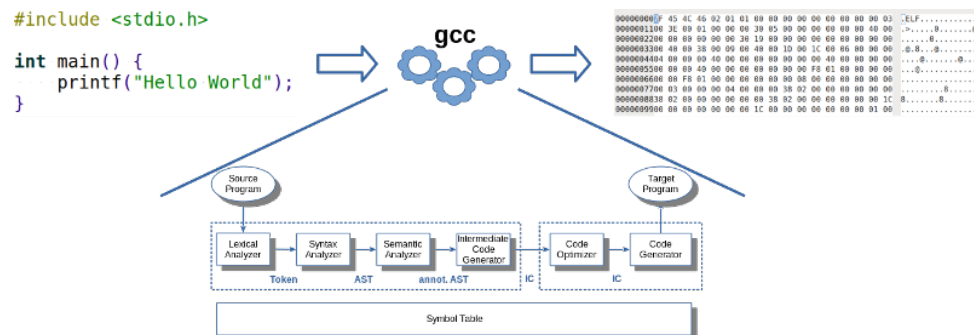
4.1.5. Wrap-Up	20
4.2. LL-Parser	21
4.2.1. Wiederholung	21
4.2.2. Motivation	21
4.2.3. Syntaxanalyse	22
4.2.4. LL(k)-Grammatiken	22
4.2.5. Wrap-Up	25
5. Sprache L-Int: Integer, Addition, Subtraktion	27
5.0.1. Teil 1:	27
5.0.2. Teil 2:	27
6. Sprache L-Expr: erweiterte Ausdrücke, Vorrangregeln	28
6.0.1. Inhalte	28
7. Sprache L-Var: Variablen, Statments, nested Scopes	29
7.0.1. Inhalt	29
8. Sprache L-If: Datentyp Boolean, Vergleiche, Kontrollstrukturen (if/else, while)	30
8.0.1. Inhalte	30
9. Sprache L-Fun: Funktionen (Definition, Aufruf)	31
9.0.1. Inhalte	31
10. Sprache L-Class: Klassen (Felder, Methoden, Objekte)	32
10.0.1. Inhalte	32
11. Sprache L-Inherit: Einfachvererbung und dynamischer Dispatch	33
11.0.1. Inhalte	33
12. Sprache L-Self: native Klassen (inkl. Vererbung)	34
12.0.1. Inhalte	34
III. Praktikum	35
13. Blatt 01: Reguläre Sprachen	37
13.1. A1.1: Sprachen von regulären Ausdrücken (1P)	37
13.2. A1.2: Bezeichner in Programmiersprachen (3P)	37
13.3. A1.3: Gleitkommazahlen in Programmiersprachen (2P)	37
13.4. A1.4: Mailadressen? (1P)	38
13.5. A1.5: Der zweitletzte Buchstabe (1P)	38
13.6. A1.6: Sprache einer regulären Grammatik (2P)	38
14. Blatt 02: CFG	39
14.1. A2.1: PDA (3P)	39
14.2. A2.2: Akzeptierte Sprache (2P)	39

14.3. A2.3: Kontextfreie Sprache (2P)	40
14.4. A2.4: Kontextfreie Grammatik (3P)	40

Teil I.

Syllabus

1. Syllabus



1.1. Kursbeschreibung

Der Compiler ist das wichtigste Werkzeug in der Informatik. In der Königsdisziplin der Informatik schließt sich der Kreis, hier kommen die unterschiedlichen Algorithmen und Datenstrukturen und Programmiersprachenkonzepte zur Anwendung.

In diesem Modul geht es um ein grundlegendes Verständnis für die wichtigsten Konzepte im Compilerbau. Wir schauen uns dazu relevante aktuelle Tools und Frameworks an und setzen diese bei der Erstellung eines kleinen Compiler-Frontends für C++ ein.

1.2. Überblick Modulinhalte

1. Lexikalische Analyse: Scanner/Lexer

- Reguläre Sprachen
- Generierung mit ANTLR

2. Syntaxanalyse: Parser

- Kontextfreie Grammatiken (CFG)
- LL-Parser (Top-Down-Parser)
- Generierung mit ANTLR

3. Semantische Analyse: Symboltabellen

- Namen und Scopes
- Typen, Klassen, Polymorphie

4. Interpreter: AST-Traversierung
5. C++ als zu verarbeitende Programmiersprache

1.3. Team

- [BC George](#)
- [Carsten Gips](#) (Sprechstunde nach Vereinbarung)
- Alesia Herbertz, Vivien Traue, Jonathan Hauer (Tutor:innen)

1.4. Kursformat

Vorlesung (2 SWS)	Praktikum (2 SWS)
Mo, 15:45 - 17:15 Uhr (Zoom)	G1: Mi, 11:30 - 13:00 Uhr (Zoom)
	G2: Mi, 14:00 - 15:30 Uhr (Zoom)
	G3: Mi, 11:30 - 13:00 Uhr (Präsenz B40)
	G4: Mi, 14:00 - 15:30 Uhr (Präsenz B40)

Durchführung der Vorlesung als *Flipped Classroom* (Carsten) bzw. als *reguläre Vorlesung* (BC). Zugangsdaten Zoom siehe [ILIAS](#).

1.5. Fahrplan

Abgabe der Post Mortems jeweils **Mittwoch bis 09:00 Uhr** im [ILIAS](#). Vorstellung der Lösung im jeweiligen Praktikum in der Abgabewoche.

Monat	Woche vom	Vorlesung (Mo)	Praktikum (Mi)	Edmonton/Minden-Meetings
Oktober	12. ...	Orga Überblick		
	19. ...	Reguläre Sprachen 1	B01	
	26. ...	Reguläre Sprachen 2 CFG	B02	
November	02. ...	LL-Parser (Theorie)		Di, 03.11., 17:00 - 18:00 Uhr (online): ANTLR + Live-Coding
	09. ...	L-Int (Teil 1)	Station 1	

Monat	Woche vom	Vorlesung (Mo)	Praktikum (Mi)	Edmonton/Minden- Meetings
Dezember	16. ...	L-Int (Teil 2)	C-Int	
	23. ...	L-Expr	C-Expr	
	30.11. ...	L-Var	Station 2	Mo, 30.11., 17:00 - 18:00 Uhr (online): Minden Presentations
	07. ...	L-If	C-Var, C-If	Mo, 07.12., 17:00 - 18:00 Uhr (online): Edmonton Presentations
Januar	14. ...	L-Fun	C-Fun	
	21. ...	Weihnachtspause		
	28. ...	Weihnachtspause		
	04. ...	L-Class	Station 3	
	11. ...	L-Inherit	C-Class, C-Inherit	
	18. ...	L-Self	C-Self, Snake	
	25. ...	Rückblick	Projektvorstellung (Video)	

1.6. Prüfungsform, Note und Credits

(Digitale) Klausur plus Studienleistung (Portfolio), 5 ECTS

- **Studienleistung:** "Portfolio" - Kriterien je Person:

1. Teilnahme an mind. zwei Edmonton/Minden-Terminen mit aktiver Beteiligung
2. Mind. vier der Übungsblätter B01..B07 erfolgreich bearbeitet
3. Abschlussvortrag zum erfolgreich bearbeiteten Mini-Projekt (B08) am Semesterende (21.01.) a 15 Minuten (pro Team)

Je Kriterium: Abgabe eines Post Mortem im ILIAS (**jede Person individuell**)

- **Gesamtnote:** (Digitale) Klausur im B40 (90 Minuten)

Hinweise

- Die Bearbeitung der Leistungen erfolgt im Team.

- Ein Team umfasst 3 Personen.
- Die Post Mortems sind individuell zu erstellen und abzugeben.
- “Aktive Beteiligung” umfasst Anwesenheit und sachbezogene Beiträge; Anwesenheit/Beteiligung werden dokumentiert.
- “Erfolgreiche Bearbeitung” eines Blattes umfasst Bearbeitung im Team, Bearbeitung aller Aufgaben des Blattes, fristgerechte Abgabe des ausreichenden Post Mortems im ILIAS. Die intensive Beschäftigung mit den Aufgaben muss erkennbar sein.
- **Post Mortem:** Jede Person beschreibt individuell(!) die Bearbeitung des jeweiligen Kriteriums bzw. die Teilnahme an den Edmonton/Minden-Meetings zurückblickend mit mind. 150 bis max. 400 Wörtern (Nutzlast; Überschriften und Links zählen nicht mit). Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein:
 1. Zusammenfassung: Was wurde gemacht bzw. was wurde auf dem Meeting besprochen?
 2. Details: Kurze Beschreibung besonders interessanter Aspekte.
 3. Reflexion: Was war der schwierigste Teil? Wie haben Sie dieses Problem gelöst?
 4. Reflexion: Was haben Sie gelernt oder (besser) verstanden?
 5. Team: Mit wem haben Sie zusammengearbeitet?
 6. Link zu Ihrem Repo mit den relevanten Artefakten (Lösung, Slides für den Vortrag, ...).

Für die Edmonton/Minden-Meetings passen Sie bitte die Punkte (1) bis (4) entsprechend inhaltlich an, (5) und (6) entfallen.

Die Post Mortems geben Sie bitte pro Person bis spätestens zur jeweiligen Deadline im [ILIAS](#) ab.

Siehe auch <https://github.com/Compiler-CampusMinden/CB-Vorlesung-Bachelor-W25/discussions/3>.

1.7. Materialien

1. “**Compilers: Principles, Techniques, and Tools**”. Aho, A. V. und Lam, M. S. und Sethi, R. und Ullman, J. D. and Bansal, S., Pearson India, 2023. ISBN 978-9-3570-5488-1. [Online](#) über die [O’Reilly-Lernplattform](#).
2. “**Crafting Interpreters**”. Nystrom, R., Genever Benning, 2021. ISBN 978-0-9905829-3-9. [Online](#).
3. “**The Definitive ANTLR 4 Reference**”. Parr, T., Pragmatic Bookshelf, 2014. ISBN 978-1-9343-5699-9. [Online](#) über die [O’Reilly-Lernplattform](#).
4. “**Writing a C Compiler**”. Sandler, N., No Starch Press, 2024. ISBN 978-1-0981-8222-9. [Online](#) über die [O’Reilly-Lernplattform](#).

1.8. Förderungen und Kooperationen

1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada)

Über das Projekt “[We CAN virtuOWL](#)” der Fachhochschule Bielefeld ist im Frühjahr 2021 eine Kooperation mit der [University of Alberta](#) (Edmonton/Alberta, Kanada) im Modul “Compilerbau” gestartet.

Wir freuen uns, auch in diesem Semester wieder drei gemeinsame Sitzungen für beide Hochschulen anbieten zu können. (Diese Termine werden in englischer Sprache durchgeführt.)

1.9. LICENSE



Unless otherwise noted, [this work](#) by [BC George](#), [Carsten Gips](#) and [contributors](#) is licensed under [CC BY-SA 4.0](#). See the [credits](#) for a detailed list of contributing projects.

Teil II.

Vorlesungsunterlagen

2. Überblick

Was ist ein Compiler? Welche Bausteine lassen sich identifizieren, welche Aufgaben haben diese?

3. Lexikalische Analyse

In der lexikalischen Analyse soll ein Lexer (auch “Scanner”) den Zeichenstrom in eine Folge von Token zerlegen. Zur Spezifikation der Token werden in der Regel reguläre Ausdrücke verwendet.

3.1. Reguläre Sprachen, Ausdrucksstärke (Teil 1)

Weitere Unterlagen

- [Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke](#)

3.1.1. Motivation

3.1.1.1. Was muss ein Compiler wohl als erstes tun?

Hier entsteht ein Tafelbild.

3.1.1.2. Themen für heute

- Lexer
- Endliche Automaten
- Reguläre Sprachen

3.1.2. Endliche Automaten

3.1.2.1. Alphabete

Def.: Ein **Alphabet** Σ ist eine endliche, nicht-leere Menge von Symbolen. Die Symbole eines Alphabets heißen *Buchstaben*.

Def.: Ein **Wort** w über einem Alphabet Σ ist eine endliche Folge von Symbolen aus Σ . ϵ ist das leere Wort. Die *Länge* $|w|$ eines Wortes w ist die Anzahl von Buchstaben, die es enthält (Kardinalität).

Def.: Eine **Sprache** L über einem Alphabet Σ ist eine Menge von Wörtern über diesem Alphabet. Sprachen können endlich oder unendlich viele Wörter enthalten.

3.1.2.2. State machine

Hier entsteht ein Tafelbild.

3.1.2.3. Deterministische endliche Automaten

Bestimmte State machines:

- Eingaben bestimmen Zustandsübergänge
- Zustandsübergänge sind eindeutig
- Es gibt Anfang(szustand) und End(zustand)e

3.1.2.4. Wie definieren wir das formal?

Hier entsteht ein Tafelbild.

3.1.2.5. Def.: Deterministischer endlicher Automat

Def.: Ein **deterministischer endlicher Automat** (DFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \rightarrow Q$, δ kann partiell sein
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.6. Beispiel

Hier entsteht ein Tafelbild.

3.1.2.7. Eingabewörter statt Buchstaben

Def.: Wir definieren $\delta^* : (Q \times \Sigma^*) \rightarrow Q$: induktiv wie folgt:

- Basis: $\delta^*(q, \epsilon) = q \ \forall q \in Q$
- Induktion: $\delta^*(q, a_1, \dots, a_n) = \delta(\delta^*(q, a_1, \dots, a_{n-1}), a_n)$

Def.: Ein DFA akzeptiert ein Wort $w \in \Sigma^*$ genau dann, wenn $\delta^*(q_0, w) \in F$.

3.1.2.8. Beispiel

Hier entsteht ein Tafelbild.

3.1.2.9. Nichtdeterministische endliche Automaten

Hier entsteht ein Tafelbild.

3.1.2.10. Def.: Nichtdeterministischer Automat

Def.: Ein **nichtdeterministischer endlicher Automat** (NFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \rightarrow Q$
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.11. Akzeptierte Sprachen

Def.: Sei A ein DFA oder ein NFA. Dann ist $L(A)$ die von A akzeptierte Sprache, d. h.

$$L(A) = \{\text{Wörter } w \mid \delta^*(q_0, w) \in F\}$$

3.1.2.12. Wozu NFAs im Compilerbau?

Pattern Matching (Erkennung von Schlüsselwörtern, Bezeichnen, ...) geht mit NFAs.

NFAs sind so nicht zu programmieren, aber:

Satz: Eine Sprache L wird von einem NFA akzeptiert $\Leftrightarrow L$ wird von einem DFA akzeptiert.

D. h. es existieren Algorithmen zur

- Umwandlung von NFAs in DFAs
- Minimierung von DFAs

3.1.3. Reguläre Sprachen

3.1.3.1. Reguläre Ausdrücke definieren Sprachen

Def.: Induktive Definition von **regulären Ausdrücken** (regex) und der von ihnen repräsentierten Sprache **L**:

- Basis:
 - ϵ und $\emptyset$ sind reguläre Ausdrücke mit $L(\epsilon) = \{\epsilon\}$, $L(\emptyset) = \emptyset$
 - Sei a ein Symbol $\Rightarrow a$ ist ein regex mit $L(a) = \{a\}$
- Induktion: Seien E, F reguläre Ausdrücke. Dann gilt:
 - $E + F$ ist ein regex und bezeichnet die Vereinigung $L(E + F) = L(E) \cup L(F)$
 - EF ist ein regex und bezeichnet die Konkatenation $L(EF) = L(E)L(F)$
 - E^* ist ein regex und bezeichnet die Kleene-Hülle $L(E^*) = (L(E))^*$
 - (E) ist ein regex mit $L((E)) = L(E)$

Vorrangregeln der Operatoren für reguläre Ausdrücke: $*$, Konkatenation, $+$

3.1.3.2. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.3. Wichtige Identitäten

Satz: Sei A ein DFA $\Rightarrow \exists$ regex R mit $L(A) = L(R)$.

Satz: Sei E ein regex $\Rightarrow \exists$ DFA A mit $L(E) = L(A)$.

3.1.3.4. Formale Grammatiken

Hier entsteht ein Tafelbild.

3.1.3.5. Formale Definition formaler Grammatiken

Def.: Eine *formale Grammatik* ist ein 4-Tupel $G = (N, T, P, S)$ aus

- N : endliche Menge von **Nichtterminalen**
- T : endliche Menge von **Terminalen**, $N \cap T = \emptyset$
- $S \in N$: **Startsymbol**
- P : endliche Menge von **Produktionen** der Form

$$X \rightarrow Y \text{ mit } X \in (N \cup T)^* N (N \cup T)^*, Y \in (N \cup T)^*$$

3.1.3.6. Ableitungen

Def.: Sei $G = (N, T, P, S)$ eine Grammatik, sei $\alpha A \beta$ eine Zeichenkette über $(N \cup T)^*$ und sei $A \rightarrow \gamma$ eine Produktion von G .

Wir schreiben: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ($\alpha A \beta$ leitet $\alpha \gamma \beta$ ab).

Def.: Wir definieren die Relation $\Rightarrow^*$ induktiv wie folgt:

- Basis: $\forall \alpha \in (N \cup T)^* \alpha \Rightarrow^* \alpha$ (Jede Zeichenkette leitet sich selbst ab.)
- Induktion: Wenn $\alpha \Rightarrow^* \beta$ und $\beta \Rightarrow \gamma$ dann $\alpha \Rightarrow^* \gamma$

Def.: Sei $G = (N, T, P, S)$ eine formale Grammatik. Dann ist $L(G) = \{\text{Wörter } w \text{ über } T \mid S \Rightarrow^* w\}$ die von G erzeugte Sprache.

3.1.3.7. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.8. Reguläre Grammatiken

Def.: Eine **reguläre (oder type-3-) Grammatik** ist eine formale Grammatik mit den folgenden Einschränkungen:

- Alle Produktionen sind entweder von der Form
 - $X \rightarrow aY$ mit $X \in N, a \in T, Y \in N$ (*rechtsreguläre Grammatik*) oder
 - $X \rightarrow Ya$ mit $X \in N, a \in T, Y \in N$ (*linksreguläre Grammatik*)
- $X \rightarrow \epsilon$ ist erlaubt

3.1.3.9. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.10. Reguläre Sprachen

Satz: Die von endlichen Automaten akzeptierte Sprachklasse, die von regulären Ausdrücken beschriebene Sprachklasse und die von regulären Grammatiken erzeugte Sprachklasse sind identisch und heißen **reguläre Sprachen**.

3.1.4. Wrap-Up

3.1.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- DFAs und NFAs
- Reguläre Ausdrücke
- Reguläre Grammatiken
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren

Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne DFAs
- k1: Ich kenne reguläre Ausdrücke
- k1: Ich kenne reguläre Grammatiken
- k2: Ich kann die Zusammenhänge und Gesetzmäßigkeiten bzgl. der oben genannten Konstrukte an einem Beispiel erklären
- k3: Ich kann für eine Fragestellung DFAs, reguläre Ausdrücke, reguläre Grammatiken entwickeln
- k3: Ich kann einen DFA entwickeln, der alle Schlüsselwörter, Namen und weitere Symbole einer Programmiersprache akzeptiert

3.2. Reguläre Sprachen, Ausdrucksstärke (Teil 2)

Weitere Unterlagen

- [Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke](#)

3.2.1. Wiederholung

3.2.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Wie sind DFAs und NFAs definiert?
- Was sind reguläre Ausdrücke?
- Was sind formale und reguläre Grammatiken?
- In welchem Zusammenhang stehen all diese Begriffe?

3.2.2. Motivation

3.2.2.1. Was haben reguläre Sprachen mit Compilern zu tun?

Hier entsteht ein Tafelbild.

3.2.2.2. Themen für heute

- Reguläre Sprachen
- Lexer
- Grenzen regulärer Sprachen

3.2.2.3. Wozu reguläre Sprachen im Compilerbau?

Reguläre Ausdrücke

- definieren Schlüsselwörter und alle weiteren Symbole einer Programmiersprache, z. B. den Aufbau von Gleitkommazahlen
- werden (oft von einem Generator) in DFAs umgewandelt
- sind die Basis des *Scanners* oder *Lexers*

3.2.3. Lexer

3.2.3.1. Ein Lexer ist mehr als ein DFA

Ein **Lexer**

- kann aus regulären Ausdrücken automatisch generiert werden
- wandelt mittels DFAs aus regulären Ausdrücken die Folge von Zeichen der Quelldatei in eine Folge von sog. Token um
- bekommt als Input eine Liste von Paaren aus regulären Ausdrücken und Tokennamen, z. B. ("while", WHILE)
- Kommentare und Strings müssen richtig erkannt werden. (Schachtelungen)
- liefert Paare von Token und deren Werte, sofern benötigt, z. B. (WHILE, _), oder (IDENTIFIER, "radius") oder (INTEGERZAHL, "334")

3.2.3.2. Wofür reichen reguläre Sprachen nicht?

Für z. B. alle Sprachen, in deren Wörtern Zeichen über eine Konstante hinaus gezählt werden müssen. Diese Sprachen lassen sich oft mit Variablen im Exponenten beschreiben, die unendlich viele Werte annehmen können.

- $a^i b^{2*i}$ ist nicht regulär

- $a^i b^{2*i}$ für $0 \leq i \leq 3$ ist regulär
- Wo finden sich die oben genannten Variablen bei einem DFA wieder?
- Warum ist die erste Sprache oben nicht regulär, die zweite aber?

3.2.3.3. Wie geht es weiter?

Ein Parser

- führt mit Hilfe des Tokenstreams vom Lexer die Syntaxanalyse durch
- basiert auf einer sog. kontextfreien Grammatik, deren Terminale die Token sind
- liefert die syntaktische Struktur in Form eines Ableitungsbaums (**syntax tree, parse tree**), bzw. einen **AST** (abstract syntax tree) ohne redundante Informationen im Ableitungsbaum (z. B. Semikolons)
- liefert evtl. Fehlermeldungen

3.2.4. Wrap-Up

3.2.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren
- Grenzen regulärer Sprachen

Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne die Aufgaben eines Lexers
- k1: Ich kenne die Zusammenhänge zwischen DFAs, regulären Ausdrücken, regulären Grammatiken und Lexern
- k2: Ich kann für eine Beispielsprache begründen, warum sie nicht mit einem der oben genannten Mechanismen beschrieben werden kann

4. Syntaktische Analyse

In der syntaktischen Analyse arbeitet ein Parser mit dem Tokenstrom, der vom Lexer kommt. Mit Hilfe einer Grammatik wird geprüft, ob gültige Sätze im Sinne der Sprache/Grammatik gebildet wurden. Der Parser erzeugt dabei den Parse-Tree. Man kann verschiedene Parser unterscheiden, beispielsweise die LL- und die LR-Parser.

4.1. CFG

Weitere Unterlagen

- [Annotierte Folien: CFG, LL-Parser](#)

4.1.1. Wiederholung

4.1.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Was ist ein Lexer?
- In welchem Zusammenhang stehen Lexer und reguläre Sprachen?
- Was können Lexer nicht?

4.1.2. Motivation

4.1.2.1. Was brauchen wir jetzt?

4.1.2.2. Themen für heute

- PDAs: mächtiger als DFAs, NFAs
- kontextfreie Grammatiken und Sprachen: mächtiger als reguläre Grammatiken und Sprachen
- DPDAs und deterministisch kontextfreie Grammatiken: die Grundlage der Syntaxanalyse im Compilerbau
- Syntaxanalyse

4.1.2.3. Einordnung: Erweiterung der Automatenklasse DFA, um komplexere Sprachen als die regulären akzeptieren zu können

Wir spendieren den DFAs einen möglichst einfachen, aber beliebig großen, Speicher, um zählen und matchen zu können. Wir suchen dabei konzeptionell die "kleinstmögliche" Erweiterung, die die akzeptierte Sprachklasse gegenüber DFAs vergrößert.

- Der konzeptionell einfachste Speicher ist ein Stack. Wir haben keinen wahlfreien Zugriff auf die gespeicherten Werte.
- Es soll eine deterministische und eine indeterministische Variante der neuen Automatenklasse geben.
- In diesem Zusammenhang wird der Stack auch Keller genannt.

4.1.2.4. Kellerautomaten (Push-Down-Automata, PDAs)

Def.: Ein Kellerautomat (PDA) $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ ist ein Septupel mit:

Q :	eine endliche Menge von Zuständen
Σ :	eine endliche Menge von Eingabesymbolen
Γ :	ein endliches Kelleralphabet
δ :	die Übergangsfunktion
	$\delta : Q \times \Sigma \cup \{\epsilon\} \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$
q_0 :	der Startzustand
$\perp \in \Gamma$:	der anfängliche Kellerinhalt, symbolisiert den leeren Keller ($\perp = \text{bottom}$)
$F \subseteq Q$:	die Menge von Endzuständen

Abbildung 4.1.: Definition eines PDAs

Ein PDA ist per Definition nichtdeterministisch und kann spontane Zustandsübergänge durchführen.

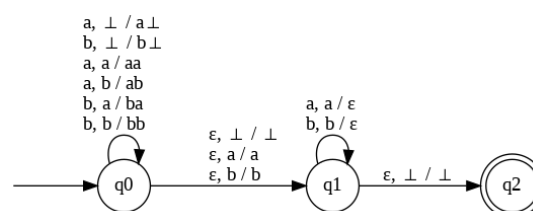
4.1.2.5. Was kann man damit akzeptieren?

Strukturen mit paarweise zu matchenden Symbolen.

Bei jedem Zustandsübergang wird ein Zeichen (oder ϵ) aus der Eingabe gelesen, ein Symbol von Keller genommen. Diese und das Eingabezeichen bestimmen den Folgezustand und eine Zeichenfolge, die auf den Stack gepackt wird. Dabei wird ein Symbol, (z. B. eines, das später mit einem Eingabesymbol zu matchen ist,) auf den Stack gepackt. Soll das automatisch vom Stack genommene Symbol auf dem Stack bleiben, muss es wieder gepusht werden.

4.1.2.6. Beispiel

Ein PDA für $L = \{ww^R \mid w \in \{a, b\}^*\}$:



4.1.2.7. Noch ein Beispiel

Hier entsteht ein Tafelbild.

4.1.2.8. Deterministische PDAs

Def. Ein PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ ist *deterministisch* $:\Leftrightarrow$

- $\delta(q, a, X)$ hat höchstens ein Element für jedes $q \in Q, a \in \Sigma$ oder ($a = \epsilon$ und $X \in \Gamma$).
- Wenn $\delta(q, a, x)$ nicht leer ist für ein $a \in \Sigma$, dann muss $\delta(q, \epsilon, x)$ leer sein.

Deterministische PDAs werden auch *DPDAs* genannt.

4.1.2.9. Der kleine Unterschied

Satz: Die von DPDAs akzeptierten Sprachen sind eine echte Teilmenge der von PDAs akzeptierten Sprachen.

Die regulären Sprachen sind eine echte Teilmenge der von DPDAs akzeptierten Sprachen.

4.1.3. Kontextfreie Grammatiken und Sprachen

4.1.3.1. Kontextfreie Grammatiken

Def. Eine *kontextfreie (cf-)* Grammatik ist ein 4-Tupel $G = (N, T, P, S)$ mit N, T, S wie in (formalen) Grammatiken und P ist eine endliche Menge von Produktionen der Form:

$X \rightarrow Y$ mit $X \in N, Y \in (N \cup T)^*$.

$\Rightarrow, \Rightarrow^*$ sind definiert wie bei regulären Sprachen. Bei cf-Grammatiken nennt man die Ableitungsbäume oft *Parse trees*.

4.1.3.2. Beispiel

$S \rightarrow a \mid S + S \mid S * S$
Hier entsteht ein Tafelbild.

Ableitungsbäume für $a + a * a$:

Hier entsteht ein Tafelbild.

4.1.3.3. Was ist hier los?

4.1.3.4. Nicht jede kontextfreie Grammatik ist eindeutig

Def.: Gibt es in einer von einer kontextfreien Grammatik erzeugten Sprache ein Wort, für das mehr als ein Ableitungsbaum existiert, so heißt diese Grammatik *mehrdeutig*. Anderenfalls heißt sie *eindeutig*.

Satz: Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.

Satz: Es gibt kontextfreie Sprachen, für die keine eindeutige Grammatik existiert.

4.1.3.5. Kontextfreie Grammatiken und PDAs

Satz: Die kontextfreien Sprachen und die Sprachen, die von PDAs akzeptiert werden, sind dieselbe Sprachklasse.

Satz: Eine von einem DPDA akzeptierte Sprache hat eine eindeutige Grammatik.

Vorgehensweise im Compilerbau: Eine (cf) Grammatik für die gewünschte Sprache definieren und schauen, ob sich daraus ein DPDA generieren lässt (automatisch).

4.1.4. Syntaxanalyse

4.1.4.1. Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

4.1.4.2. Ziele der Syntaxanalyse

- Bestimmung der syntaktischen Struktur eines Programms
- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltabelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche

4.1.4.3. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (= Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

4.1.5. Wrap-Up

4.1.5.1. Das sollen Sie mitnehmen

- Die Struktur von gängigen Programmiersprachen lässt sich nicht mit regulären Ausdrücken beschreiben und damit nicht mit DFAs akzeptieren.
- Das Automatenmodell der DFAs wird um einen endlosen Stack erweitert, das ergibt PDAs.
- Kontextfreie Grammatiken (CFGs) erweitern die regulären Grammatiken.
- PDAs akzeptieren kontextfreie Sprachen.
- Deterministisch parsbare Sprachen haben eine eindeutige kontextfreie Grammatik, aber nicht für jede eindeutige kontextfreie Grammatik lässt sich ein deterministischer PDA finden.

- Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.
- Syntaxanalyse wird mit (möglichst deterministisch) kontextfreien Grammatiken durchgeführt.
- In der Praxis werden aus kontextfreien Grammatiken Parser automatisch generiert.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne PDAs
- k1: Ich kenne deterministische PDAs
- k1: Ich kenne kontextfreie Grammatiken
- k1: Ich kenne deterministisch kontextfreie Grammatiken
- k2: Ich kann den Zusammenhang zwischen PDAs und kontextfreien Grammatiken an einem Beispiel erklären
- k2: Ich kann PDAs entwickeln
- k2: Ich kann kontextfreie Grammatiken entwickeln

4.2. LL-Parser

Weitere Unterlagen

- [Annotierte Folien: LL-Parser](#)

4.2.1. Wiederholung

4.2.1.1. PDAs und kontextfreie Grammatiken

- Warum reichen uns DFAs nicht zum Matchen von Eingabezeichen?
- Wie können wir sie minimal erweitern?
- Sind PDAs deterministisch?
- Wie sind kontextfreie Grammatiken definiert?
- Sind kontextfreie Grammatiken eindeutig?

4.2.2. Motivation

4.2.2.1. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (=Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

4.2.2.2. Themen für heute

- Automatische Generierung von Top-Down-Parsern aus LL-Grammatiken

4.2.3. Syntaxanalyse

4.2.3.1. Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

Syntaxanalyse ist die Bestimmung, ob Eingabedaten einer vorgegebenen Syntax entsprechen.

Diese vorgegebene Syntax wird im Compilerbau mit einer kontextfreien Grammatik beschrieben und mit einem sogenannten **Parser** analysiert.

Wir beschäftigen uns heute mit LL-Parsing, mit dem man eine Teilmenge der eindeutigen kontextfreien Grammatiken syntaktisch analysieren kann.

Der Ableitungsbaum wird von oben nach unten aufgebaut.

4.2.3.2. Ziele der Syntaxanalyse

- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- evtl. Fehlerkorrektur
- Bestimmung der syntaktischen Struktur eines Programms
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltabelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche.

4.2.4. LL(k)-Grammatiken

4.2.4.1. First-Mengen

$$S \rightarrow A \mid B \mid C$$

Welche Produktion nehmen?

Wir brauchen die “terminalen k-Anfänge” von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der Vorschautoken.

Def.: Wir definieren *First* - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \setminus T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \xrightarrow{*} w, \text{ mit } w \in T^*, First_k(w) = \{v\}\}$

4.2.4.2. Linksableitungen

Def.: Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \xRightarrow{*}_l \beta$.

4.2.4.3. LL(k)-Grammatiken

Def.: Eine kontextfreie Grammatik $G = (N, T, P, S)$ ist genau dann eine $LL(k)$ -Grammatik, wenn für alle Linksableitungen der Form:

$$S \xRightarrow{*}_l wA\gamma \Rightarrow_l w\alpha\gamma \xRightarrow{*}_l wx$$

und

$$S \xRightarrow{*}_l wA\gamma \Rightarrow_l w\beta\gamma \xRightarrow{*}_l wy$$

mit $(w, x, y \in T^*, \alpha, \beta, \gamma \in (N \cup T)^*, A \in N)$ und $First_k(x) = First_k(y)$ gilt:

$$\alpha = \beta$$

4.2.4.4. LL(1)-Grammatiken

Hier entsteht ein Tafelbild.

4.2.4.5. LL(k)-Sprachen

Die von $LL(k)$ -Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von $LL(k)$ -Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von $LL(k+1)$ -Grammatiken erzeugten Sprachen.

Für eine kontextfreie Grammatik G ist nicht entscheidbar, ob es eine $LL(1)$ -Grammatik G' gibt mit $L(G) = L(G')$.

In der Praxis reichen $LL(1)$ -Grammatiken oft. Hier gibt es effiziente Parsergeneratoren (hier: ANTLR), deren Eingabe eine $LL(k)$ - (meist $LL(1)$ -) Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellen-gesteuerten Parsers generieren.

4.2.4.6. Algorithmus: Konstruktion einer LL-Parsertabelle

Eingabe: Eine Grammatik $G = (N, T, P, S)$

Ausgabe: Eine Parsertabelle P

```

for each production  $X \rightarrow \alpha$ 
  for each  $a \in First(\alpha)$ 
    add  $X \rightarrow \alpha$  to  $P[X, a]$ 

  if  $\epsilon \in First(\alpha)$ 
    for each  $b \in Follow(\alpha)$ 
      add  $X \rightarrow \alpha$  to  $P[X, b]$ 
      if  $\epsilon \in First(\alpha)$  and  $\perp \in Follow(X)$ 
        add  $X \rightarrow \alpha$  to  $P[X, \perp]$ 

```

Abbildung 4.2.: Algorithmus zur Generierung einer LL-Parsertabelle

Hier ist $\perp$ das Endezeichen des Inputs. Statt $First_1(\alpha)$ wird oft nur $First(\alpha)$ geschrieben.

4.2.4.7. LL-Parsertabellen

Hier entsteht ein Tafelbild.

4.2.4.8. LL-Parsertabellen

Rekursive Programmierung bedeutet, dass das Laufzeitsystem einen Stack benutzt. Diesen Stack kann man auch "selbst programmieren", d. h. einen PDA implementieren. Dabei wird ebenfalls die oben genannte Tabelle zur Bestimmung der nächsten anzuwendenden Produktion benutzt. Der Stack enthält die zu erwartenden Eingabezeichen, wenn immer eine Linksableitung gebildet wird. Diese Zeichen im Stack werden mit dem Input gematcht.

4.2.4.9. Algorithmus: Tabellengesteuertes LL-Parsen mit einem PDA

Eingabe: Eine Grammatik $G = (N, T, P, S)$, eine Parsertabelle P mit " $w \perp$ " als initialem Kellerinhalt

Ausgabe: Wenn $w \in L(G)$, eine Linksableitung von w , Fehler sonst

```

a = next_token()
X = top of stack // entfernt X vom Stack

while X  $\neq$   $\perp$ 

    if X = a
        a = next_token()

    else if X  $\in$  T
        error

    else if P[X,a] leer
        error

    else if P[X,a] =  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
        process_production( $X \rightarrow Y_1 Y_2 \dots Y_k$ )
        push( $Y_1 Y_2 \dots Y_k$ ) //  $Y_1$  = top of stack

X = top of stack

```

Abbildung 4.3.: Algorithmus zum tabellengesteuerten LL-Parsen

4.2.4.10. Ergebnisse der Syntaxanalyse

- eventuelle Syntaxfehler mit Angabe der Fehlerart und des -Ortes
- Fehlerkorrektur
- Format für die Weiterverarbeitung:
 - Ableitungsbaum oder Syntaxbaum oder Parse Tree
 - abstrakter Syntaxbaum (AST): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. ;, Klammern, manche Schlüsselwörter, . . .)
- Symboltabelle

4.2.5. Wrap-Up

4.2.5.1. Wrap-Up

- Syntaxanalyse wird mit deterministisch kontextfreien Grammatiken durchgeführt.
- Eine Teilmenge der dazu gehörigen Sprachen lässt sich top-down parsen.
- Ein effizienter LL(k)-Parser realisiert einen DPDA und kann automatisch aus einer LL(k)-Grammatik generiert werden.
- Der Parser liefert in der Regel einen abstrakten Syntaxbaum.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne die Top-Down-Analyse
- k1: Ich kenne LL-Parser
- k2: Ich kann den algorithmischen Ablauf von LL-Parsern an einem Beispiel erklären

5. Sprache L-Int: Integer, Addition, Subtraktion

5.0.1. Teil 1:

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- AST, InterpreterLInt

5.0.2. Teil 2:

- Wiederholung ANTLR, Visitor mit Rückgabe (zustandslos), ANTLR-Grammatik
- Handgeschriebener Lexer, RD-Parser

6. Sprache L-Expr: erweiterte Ausdrücke, Vorrangregeln

6.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ANTLR: Vorrang mit gestuften Regeln vs. ANTLR4-Precedence
- RD+Pratt
- InterpreterLExpr

7. Sprache L-Var: Variablen, Statments, nested Scopes

7.0.1. Inhalt

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- Nested Scopes, Resolving
- ResolverLVar, InterpreterLVar

8. Sprache L-If: Datentyp Boolean, Vergleiche, Kontrollstrukturen (if/else, while)

8.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLIf, TypeCheckerLIf, InterpreterLIf
- Nano-Pass mit Desugaring: for, +=, ... (??)

9. Sprache L-Fun: Funktionen (Definition, Aufruf)

9.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLFun, TypeCheckerLFun, InterpreterLFun
- native Funktionen vs. gepackte Funktionen

10. Sprache L-Class: Klassen (Felder, Methoden, Objekte)

10.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- Klassen als Environment für Methoden, Instanzen als Environment für Felder, Auflöse-Strategien
- ResolverLClass, TypeCheckerLClass, InterpreterLClass

11. Sprache L-Inherit: Einfachvererbung und dynamischer Dispatch

11.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLInherit, TypeCheckerLInherit, InterpreterLInherit

12. Sprache L-Self: native Klassen (inkl. Vererbung)

12.0.1. Inhalte

- Einbinden von nativen Klassen
- InterpreterLSelf, TypeCheckerLSelf/ResolverLSelf
- Vorbereitung für Snake

Teil III.

Praktikum

Hier finden Sie die Übungsblätter.

13. Blatt 01: Reguläre Sprachen

13.1. A1.1: Sprachen von regulären Ausdrücken (1P)

Welche Sprache wird von dem folgenden regulären Ausdruck beschrieben?

$$a + a(a + b)^* a$$

13.2. A1.2: Bezeichner in Programmiersprachen (3P)

Betrachten Sie eine Programmiersprache, in der die Bezeichner (= Namen für Variablen, Funktionen, Klassen, Methoden, ...) folgenden Aufbau haben:

- Alle Variablenamen beginnen mit **V** oder **v**
- Handelt es sich um globale Variablen, beginnen Sie mit **V**, lokale beginnen mit **v**
- Funktions- und Methodenparameter beginnen mit **p**, Klassenparameter (bei der Definition von Vererbung) beginnen mit **P**
- Weitere Bezeichner müssen mit einem Buchstaben (a-z, A-Z) beginnen
- Die folgenden Zeichen dürfen Buchstaben, Ziffern und ein Unterstrich sein
- Bezeichner dürfen nicht mit einem Unterstrich enden
- Alle Bezeichner müssen aus mindestens zwei Zeichen bestehen

Entwickeln Sie einen regulären Ausdruck, der den Aufbau der Bezeichner beschreibt. Beachten Sie, dass Ihr regex alle zulässigen Bezeichner beschreiben muss, aber keinen einzigen unzulässigen beschreiben darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom regex gematcht werden.

Entwickeln Sie einen DFA, der diese Bezeichner akzeptiert. Beachten Sie, dass Ihr DFA alle zulässigen Bezeichner akzeptieren muss, aber keinen einzigen unzulässigen akzeptieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom Automaten zeichenweise gelesen und akzeptiert werden.

Entwickeln Sie eine reguläre Grammatik, die diese Bezeichner generiert. Beachten Sie, dass Ihre Grammatik alle zulässigen Bezeichner generieren können muss, aber keinen einzigen unzulässigen generieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie die Ableitungsbäume dazu.

13.3. A1.3: Gleitkommazahlen in Programmiersprachen (2P)

Recherchieren Sie zunächst den Aufbau von Gleitkommazahlen in Python und Java.

Erstellen Sie für jede der beiden Programmiersprachen reguläre Ausdrücke, DFAs und reguläre Grammatiken wie in Aufgabe A1.2. Verifizieren Sie Ihre Lösungen wie in Aufgabe A1.2. Vorgaben, die sich auf Längen oder Werte von Teilen der Zahlen beziehen, ignorieren Sie bitte.

13.4. A1.4: Mailadressen? (1P)

Warum ist der folgende regex ungeeignet für die Verarbeitung von Mailadressen?

$(a - z)^+ @ (a - z) . (a - z)$

Bitte beachten Sie, dass die Schreibweise a-z nicht unserer Definition genügt. Eigentlich müsste jedes Zeichen aufgeführt werden:

$a + b + c + c + \dots + z$ ist besser, aber immer noch nicht richtig. Warum?

Anmerkung: Diese Darstellung wird ab jetzt akzeptiert.

Verbessern Sie den gegebenen regulären Ausdruck.

13.5. A1.5: Der zweitletzte Buchstabe (1P)

Entwickeln Sie einen DFA, der nur Wörter über $\Sigma = \{1, 2, 3\}$ akzeptiert, deren zweitletztes Zeichen dasselbe ist wie das zweite.

13.6. A1.6: Sprache einer regulären Grammatik (2P)

Welche Sprache generiert die folgende Grammatik?

$$S \rightarrow aA$$

$$A \rightarrow dB \mid bA \mid cA$$

$$B \rightarrow aC \mid bC \mid cA$$

$$C \rightarrow \epsilon$$

Können Sie einen regulären Ausdruck oder einen DFA dafür angeben?

14. Blatt 02: CFG

14.1. A2.1: PDA (3P)

Erstellen Sie einen deterministischen PDA, der die Sprache

$$L = \{w \in \{a, b, c\}^* \mid w \text{ hat doppelt so viele a's wie c's}\}$$

akzeptiert.

Beschreiben Sie Schritt für Schritt, wie der PDA die Eingaben *bcaba* und *bccac* abarbeitet.

14.2. A2.2: Akzeptierte Sprache (2P)

Ist der folgenden PDA deterministisch? Warum (nicht)?

q_4 sei der akzeptierende Zustand.

$$\begin{aligned}\delta(q_0, a, \perp) &= (q_0, A \perp) \\ \delta(q_0, a, A) &= (q_0, AA) \\ \delta(q_0, b, A) &= (q_1, BA) \\ \delta(q_1, b, B) &= (q_1, BB) \\ \delta(q_1, c, B) &= (q_2, \epsilon) \\ \delta(q_2, c, B) &= (q_2, \epsilon) \\ \delta(q_2, d, A) &= (q_3, \epsilon) \\ \delta(q_3, d, A) &= (q_3, \epsilon) \\ \delta(q_3, d, A) &= (q_3, AA) \\ \delta(q_3, \epsilon, \perp) &= (q_4, \epsilon)\end{aligned}$$

Zeichnen Sie den Automaten. Geben Sie das 7-Tupel des PDA an. Welche Sprache akzeptiert er?

14.3. A2.3: Kontextfreie Sprache (2P)

Welche Sprache generiert die folgende kontextfreie (Teil-) Grammatik?

$$G = (\{\text{Statement, Condition, } \dots\}, \{\text{"if", "else", } \dots\}, P, \text{Statement})$$

mit

$$P = \left\{ \begin{array}{l} \text{Statement} \rightarrow \text{"if" Condition Statement} \mid \text{"if" Condition Statement "else" Statement} \\ \text{Condition} \rightarrow \dots \end{array} \right\}$$

Ist die Grammatik mehrdeutig? Warum (nicht)?

14.4. A2.4: Kontextfreie Grammatik (3P)

Entwickeln Sie eine kontextfreie Grammatik für die Sprache

$$L = \{a^i b^j c^k \mid i = j \vee j = k\}$$

Zeigen Sie, dass die Grammatik mehrdeutig ist. Entwickeln Sie einen PDA für diese Sprache.

Quellen

Aho, A. V., M. S. Lam, R. Sethi, J. D. Ullman, und S. Bansal. 2023. *Compilers: Principles, Techniques, and Tools, Updated 2nd Edition by Pearson*. Pearson India. <https://learning.oreilly.com/library/view/compilers-principles-techniques/9789357054881/>.

Hopcroft, J. E., R. Motwani, und J. D. Ullman. 2003. *Einführung in die Automatentheorie, formale Sprachen und Komplexitätstheorie*. I theoretische informatik. Pearson Education Deutschland GmbH.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. <https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/>.

Torczon, L., und K. Cooper. 2012. *Engineering a Compiler*. Morgan Kaufmann. <https://learning.oreilly.com/library/view/engineering-a-compiler/9780080916613/>.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: 16f1ea4 2026-09-04 lecture: add dummy starter for incremental languages